



## Rapport MTR

**Sujet :** Refonte d'une Application de jeu vidéo



**SNAKE GAME**

**Rédactrices :** Mélissa LATEB & Anaïs FREJABUE & Sirine REYS

**Formateurs :** Binh-Minh BUI-XUAN & Arthur ESCRIOU

**Module :** Moteur temps réel (MTR)

**Année :** 2023/2024

# **Table des matières**

|                                            |           |
|--------------------------------------------|-----------|
| <b>1. Introduction</b>                     | <b>3</b>  |
| 1.1 Membres de l'équipe                    | 3         |
| 1.2 Objectifs                              | 3         |
| 1.3 Gestion de projet                      | 3         |
| <b>2. Analyse Fonctionnelle</b>            | <b>4</b>  |
| 2.1 User story - Jouer une Partie          | 4         |
| 2.2 Wireframe                              | 4         |
| 2.3 Technologies Utilisées                 | 6         |
| 2.4 Bilan des ressources déployées         | 6         |
| 2.5 Détail des Tâches                      | 6         |
| <b>3. Partie Technique</b>                 | <b>8</b>  |
| 3.1 Architecture Générale de l'Application | 8         |
| 3.2 Présentation de la Couche Client       | 9         |
| 3.3 Présentation de la Couche Data         | 10        |
| 3.4 Présentation de la Couche Serveur      | 11        |
| <b>4. Conclusion</b>                       | <b>13</b> |

# 1. Introduction

## 1.1 Membres de l'équipe

Mélissa Lateb : Développeuse

Anaïs Frejabue : Développeuse

Sirine Reys : Développeuse

## 1.2 Objectifs

L'objectif de ce projet est de développer un clone du jeu vidéo "Snake", un jeu vidéo connu de type "old school". Nous avons choisi "Snake" pour sa simplicité et son accessibilité, ce qui permet de se concentrer sur l'implémentation des fonctionnalités de base et d'explorer les concepts fondamentaux de la programmation de jeux vidéo.

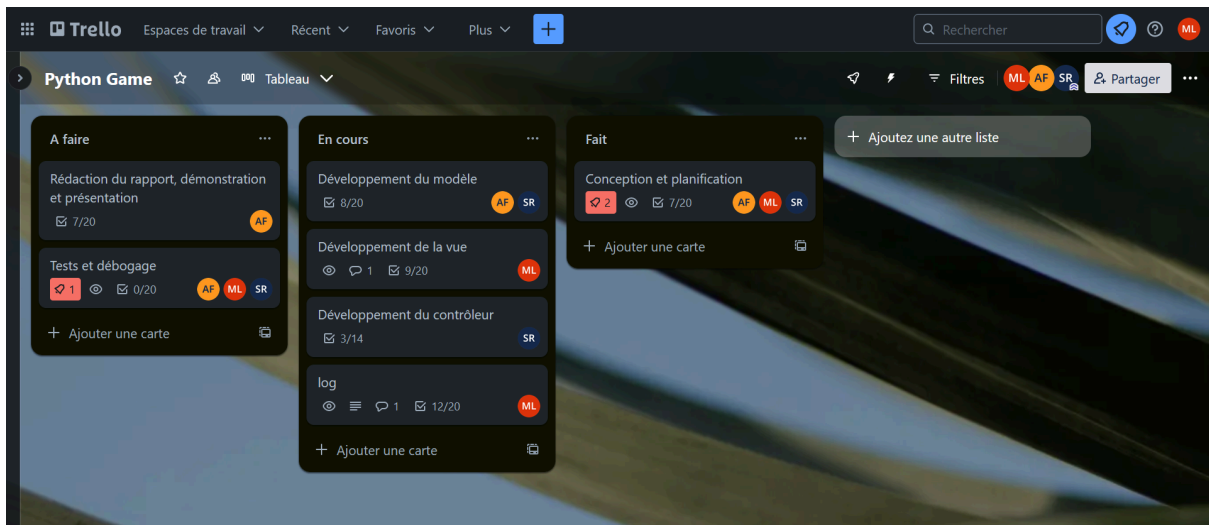
## 1.3 Gestion de projet



Afin d'échanger à distance avec l'équipe nous avons privilégié les plateformes Google meet et discord. Cela nous a permis de valider les tâches réalisées et partager notre avancée.



Pour la planification des tâches et la répartition par personne du travail à effectuer nous avons optés pour l'outil de gestion de projet Trello



Afin de centraliser le développement et le code et pouvoir collaborer nous avons opté pour github. Voici le lien du dépôt git :

<https://github.com/melissalateb/MTR-Projet-Snake>

## 2. Analyse Fonctionnelle

### 2.1 User story - Jouer une Partie



**Acteur Principal** : Joueur

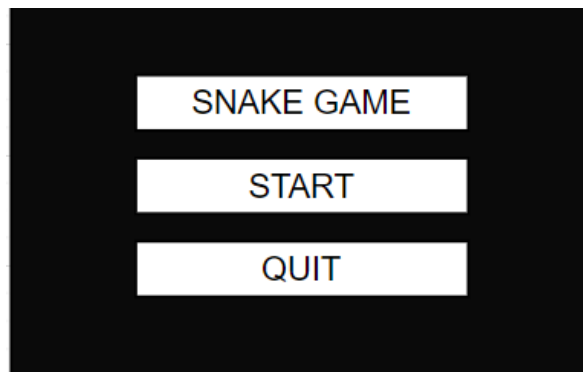
**Scénario Principal** : Le joueur démarre le jeu, contrôle le serpent à l'aide des touches directionnelles, mange des aliments pour grandir et évite les collisions avec son propre corps.

**Pré-condition** : Le jeu est lancé et le menu principal est affiché.

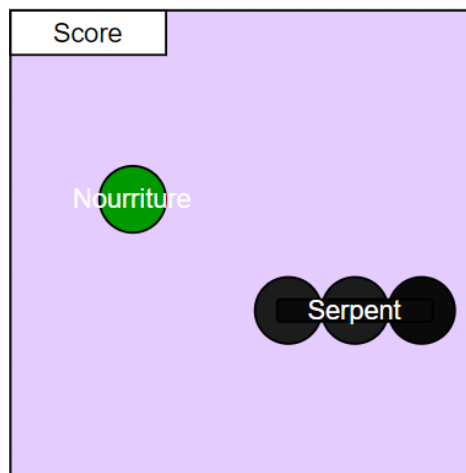
**Post-condition** : Le score du joueur est affiché tout au long de la partie.

## 2.2 Wireframe

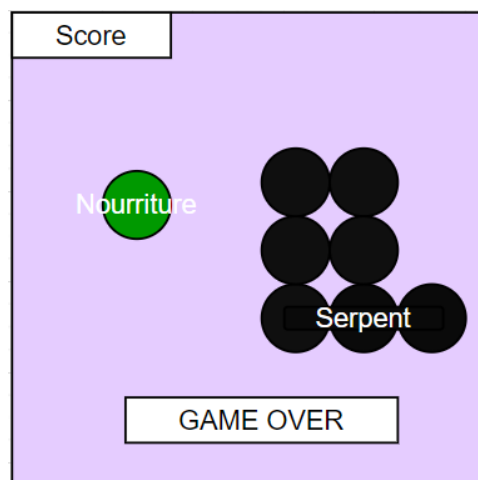
### Ecran d'accueil



### Jeu en cours



### Ecran de fin (Game Over)



## 2.3 Technologies Utilisées

| Technologie                                                                        | Utilisation                        | Justification                                                                                                                                                                          |
|------------------------------------------------------------------------------------|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|   | Création du wireframe              | Simple d'utilisation, facile à manipuler et personnaliser. Outil connu et utilisé sur d'autres projets réalisés                                                                        |
|   | Langage de programmation principal | Python est un langage simple et puissant, idéal pour des projets de prototypage rapide comme ce jeu                                                                                    |
|  | Bibliothèque pour les jeux         | Pygame est une bibliothèque Python spécialement conçue pour la création de jeu 2D, offrant des fonctionnalités faciles à utiliser pour le rendu graphique et la gestion des événements |

## 2.4 Bilan des ressources déployées

| Tâche                                                          | Durée (jours) |
|----------------------------------------------------------------|---------------|
| Conception et planification                                    | 0.25          |
| Développement du modèle                                        | 0.5           |
| Développement de la vue                                        | 0.75          |
| Développement du contrôleur                                    | 0.5           |
| Tests et débogage                                              | 0.5           |
| Rédaction du rapport<br>Vidéo de démonstration<br>Présentation | 0.5           |

## 2.5 Détail des Tâches

### 1. Conception et planification (0.25j) :

- Analyse des exigences du projet
- Conception du wireframe avec Draw.io
- Planification des étapes de développement

### 2. Développement du modèle (0.5j) :

- Implémentation de la classe **Snake** pour gérer la logique du serpent
- Implémentation de la classe **Food** pour gérer la nourriture
- Création de la classe **GameModel** pour intégrer les éléments du jeu

### 3. Développement de la vue (0.75j) :

- Mise en place de la classe **GameView** pour gérer l'affichage graphique
- Intégration des éléments graphiques pour le serpent, la nourriture, et le score

### 4. Développement du contrôleur (0.5j) :

- Création de la classe **GameController** pour gérer les interactions utilisateur
- Gestion des événements clavier pour contrôler le serpent
- Intégration de la boucle de jeu principale

### 5. Tests et débogage (0.5j) :

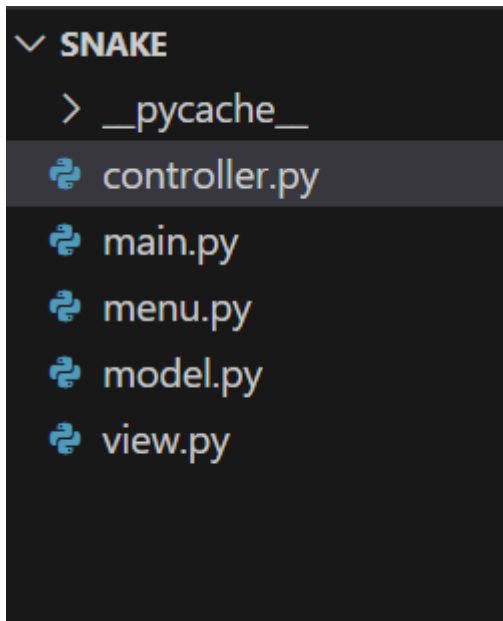
- Exécution de tests pour vérifier le bon fonctionnement des fonctionnalités implémentées
- Débogage des problèmes rencontrés lors des tests
- Ajout de logger pour les étapes de chaque jeu

### 6. Documentation, vidéo de démonstration et présentation (0.5j) :

- Ajout des commentaires du code et rédaction du rapport du projet
- Explication des algorithmes et des choix de conception
- Préparation de la présentation et la démonstration vidéo.  
Résumé du projet et des points essentiels dans la présentation.

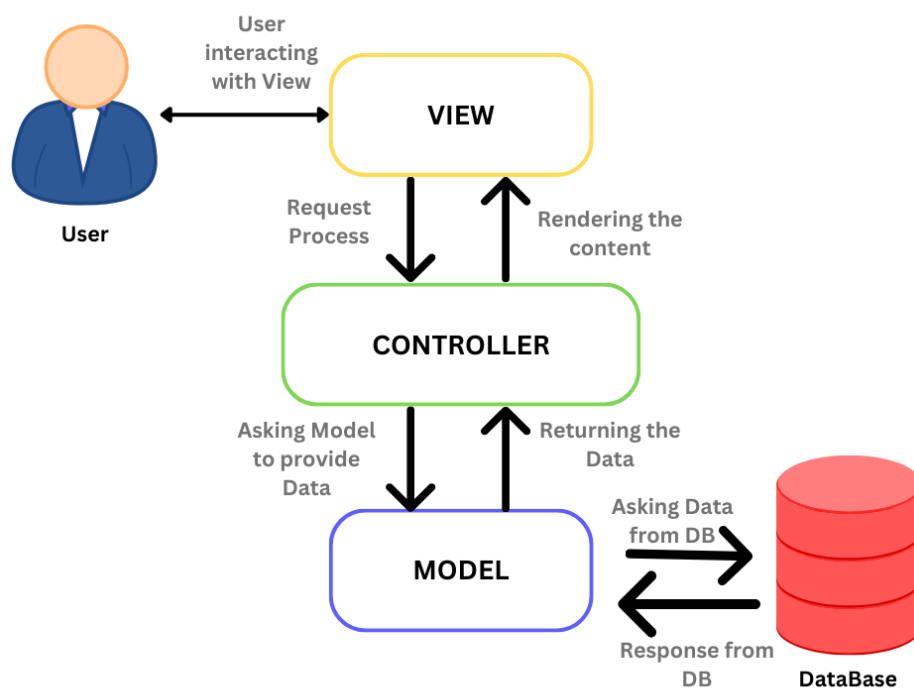
## 3. Partie Technique

### 3.1 Architecture Générale de l'Application



L'application est structurée selon le modèle MVC (Modèle-Vue-Contrôleur) :

- **Modèle** : Contient la logique du jeu (serpent, nourriture, score).
- **Vue** : Gère l'affichage graphique (dessin du serpent, de la nourriture, du score).
- **Contrôleur** : Gère les interactions utilisateur (entrée clavier, démarrage et fin de jeu).

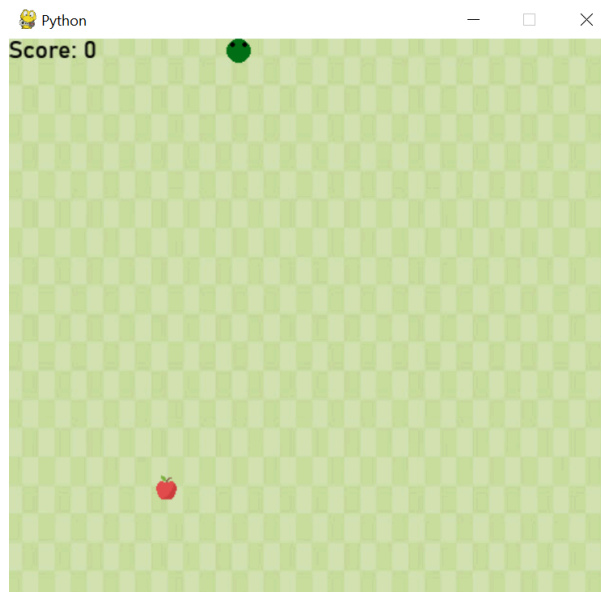


## 3.2 Présentation de la Couche Client

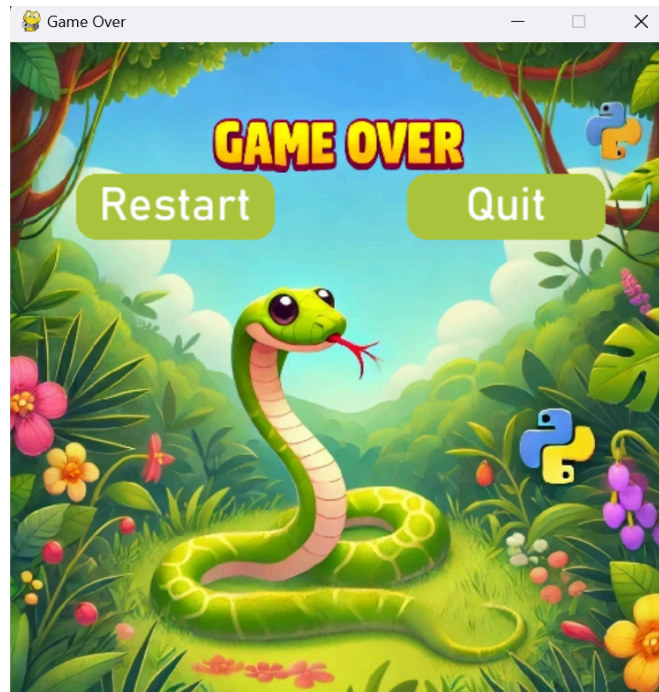
- **Menu Principal** : Affichage initial avec options pour démarrer une nouvelle partie ou quitter le jeu.



- **Jeu en cours** : Affichage du jeu avec le serpent, la nourriture, et le score.



- **Écran de Fin de Jeu** : Affichage du message de fin de jeu avec options pour démarrer ou quitter.



### 3.3 Présentation de la Couche Data

Le jeu ne nécessite pas de stockage persistant des données. Toutes les données (positions du serpent et de la nourriture, score) sont stockées en mémoire et réinitialisées à chaque partie.

## 3.4 Présentation de la Couche Serveur

### 1. Déplacement du Serpent

```
ALGORITHM MoveSnake
  INPUT: current_position (x, y), direction (dx, dy), growing
  OUTPUT: new_position, updated_body

  // Calcul de la nouvelle position de la tête
  new_head <- (x + dx, y + dy)

  IF growing > 0 THEN
    // Ajouter la nouvelle tête sans supprimer le dernier segment
    updated_body <- [new_head] + body
    growing <- growing - 1
  ELSE
    // Ajouter la nouvelle tête et supprimer le dernier segment
    updated_body <- [new_head] + body[0 : len(body) - 1]
  END IF

  RETURN new_head, updated_body
END ALGORITHM
```

### 2. Changement de Direction

```
ALGORITHM ChangeDirection
  INPUT: current_direction (dx, dy), new_direction (ndx, ndy)
  OUTPUT: updated_direction

  // Vérifier que les directions ne sont pas opposées
  IF (dx * ndx == 0) AND (dy * ndy == 0) THEN
    updated_direction <- new_direction
  ELSE
    updated_direction <- current_direction
  END IF

  RETURN updated_direction
END ALGORITHM
```

### 3. Enveloppement

```
ALGORITHM WrapPosition
  INPUT: current_position (x, y), width, height
  OUTPUT: wrapped_position

  // Calcul de la nouvelle position enveloppée
  new_x <- x % width
  new_y <- y % height
  wrapped_position <- (new_x, new_y)

  RETURN wrapped_position
END ALGORITHM
```

### 4. Génération Aléatoire de Nourriture

```
ALGORITHM SpawnFood
  INPUT: width, height
  OUTPUT: food_position

  // Génération d'une nouvelle position aléatoire pour la nourriture
  new_x <- RANDOM(0, width - 1)
  new_y <- RANDOM(0, height - 1)
  food_position <- (new_x, new_y)

  RETURN food_position
END ALGORITHM
```

### 5. Vérification de la collision avec le corps

```
ALGORITHM CheckSelfCollision
  INPUT: body
  OUTPUT: is_collision

  // Vérifie si le serpent se mord la queue, uniquement si sa taille est supérieure à 2
  IF LENGTH(body) > 2 THEN
    is_collision <- head IN body[1:]
  ELSE
    is_collision <- FALSE
  END IF

  RETURN is_collision
END ALGORITHM
```

## 4. Conclusion

La version de “Python Game” livrée ce jour est la version 1 contenant les fonctionnalités essentielles. Nous avons prévu d'ajouter plusieurs fonctionnalités en Version 2, telles que la gestion des niveaux avec des obstacles fixes et dynamiques. Il serait également pertinent de permettre la création de comptes utilisateurs pour stocker des données comme le meilleur score et l'historique des parties.

Ce projet de développement consistant à créer un clone du jeu vidéo old school “Snake” a été à la fois ludique et enrichissant. Il nous a permis de découvrir et d'appréhender des concepts fondamentaux de la programmation de jeux vidéo tels que la gestion des entrées utilisateur et le modèle MVC, le rendu graphique et la logique de jeu.

La simplicité et la puissance de Python et Pygame se sont révélées être des atouts majeurs, facilitant la transformation de nos idées en une expérience de jeu dynamique. Ce projet a été une occasion précieuse d'explorer les mécanismes du développement de jeux tout en stimulant notre créativité.

Nous sommes fiers de ce que nous avons accompli et impatients de continuer à améliorer et à étendre notre jeu avec de nouvelles fonctionnalités et des améliorations basées sur les retours des utilisateurs.